Nested Learning: The Illusion of Deep Learning Architectures

巢狀學習：深度學習架構的幻象

Authors: Ali Behrouz, Meisam Razaviyayn, Peiling Zhong, Vahab Mirrokni (Google Research)

報告人 (Presenter): Chang, Chia-Kai (張家愷)

The Core Challenge: The Static Nature of LLMs
核心挑戰：大型語言模型的靜態本質

Large Language Models (LLMs) are powerful, yet largely **static** after deployment. Their knowledge is frozen after the pre-training phase.
大型語言模型（LLMs）功能強大，但在部署後基本上是**靜態的**。其知識在預訓練階段結束後即被凍結。

The "Anterograde Amnesia" Analogy
「順行性遺忘症」的比喻

LLMs behave like a person with anterograde amnesia:
LLM 的行為如同患有順行性遺忘症的病人：

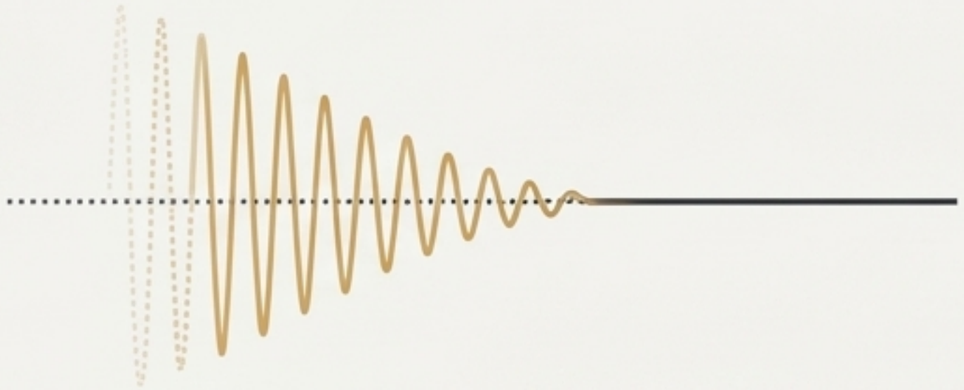
1. **Intact Old Memories**: They retain vast knowledge from their training data (the "long past").
   舊記憶完好無損：保留從訓練數據中獲得的龐大知識（「遙遠的過去」）。
2. **Inability to Form New Memories**: They cannot form new long-term memories from new information encountered in their context window.
   無法形成新記憶：無法從其脈絡視窗中遇到的新資訊形成新的長期記憶。

The model's parameters (long-term memory) are not updated by new context.
模型的參數（長期記憶）不會因新的脈絡而更新。

Inspiration: Human Brain's Memory Consolidation
靈感來源：人腦的記憶鞏固機制

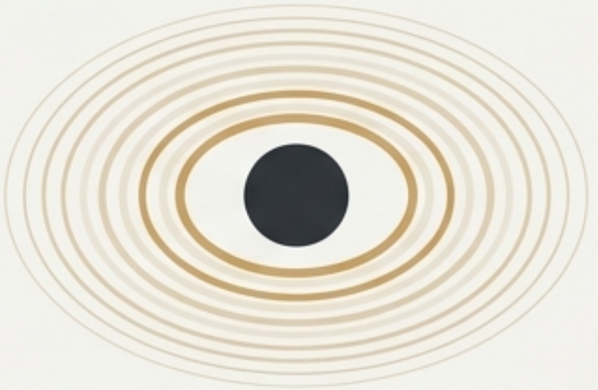
The brain efficiently manages continual learning through neuroplasticity and two key memory consolidation processes.
大腦透過神經可塑性與兩種關鍵的記憶鞏固過程，有效地管理持續學習。

1. Online (Synaptic) Consolidation | 線上 (突觸) 鞏固



- A **rapid process** occurring immediately after learning.
  學習後立即發生的**快速過程**。
- Stabilizes new, fragile memory traces and begins their transfer from short-term to long-term storage.
  穩定脆弱的新記憶痕跡，並開始將其從短期儲存轉移至長期儲存。
- *This is the phase missing in current LLMs.*
  這是當前 LLM 所缺失的階段。

2. Offline (Systems) Consolidation | 離線 (系統) 鞏固



- A **slower process**, often during sleep.
  一個**較慢的過程**，通常在睡眠期間發生。
- Replays, strengthens, and reorganizes memories for permanent storage.
  重播、強化並重組記憶，以進行永久儲存。

A New Paradigm: Nested Learning (NL)

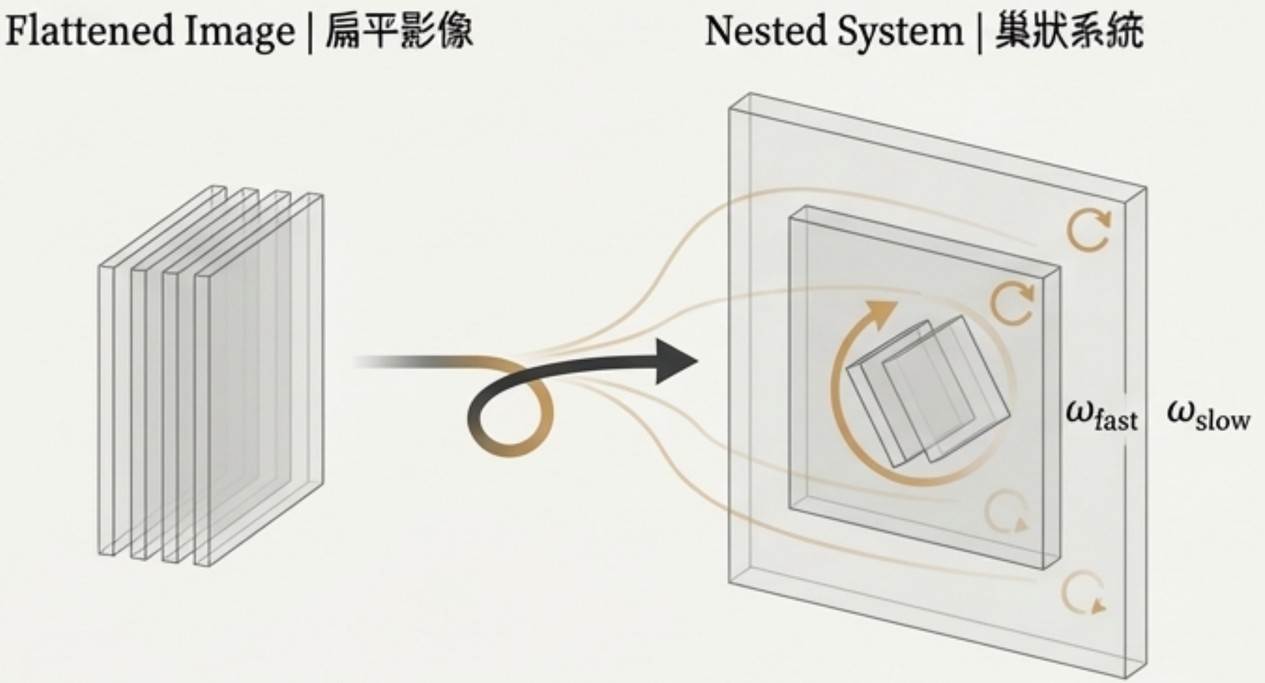
新範式：巢狀學習（NL）

NL represents a model and its training as a set of nested, multi-level, and/or parallel optimization problems.
巢狀學習 (NL) 將模型及其訓練過程，表述為一組巢狀的、多層次的、且/或平行的優化問題。

Key Idea | 核心思想

Instead of a single, flat architecture, NL views a model as an integrated system where different components learn at different speeds.
NL 不將模型視為單一的扁平架構，而是一個整合系統，其中不同組件以不同速度學習。

- Each component has its own "context flow" (脈絡流) and update frequency (更新頻率).
- This reveals the hidden computational depth within existing models. 揭示現有模型中隱藏的計算深度。
- It provides a path to design more expressive learning algorithms with more "levels (層級)".



Fundamental Unit: Learning as Associative Memory
基本單元：視學習為聯想記憶

NL redefines memory and learning from a neuropsychology perspective: NL 從神經心理學角度重新定義記憶與學習：

- **Memory (記憶):** A neural update caused by an input. 由輸入引發的神經元更新。
- **Learning (學習):** The process for acquiring *effective and useful* memory. 獲取*有效且有用*的記憶之過程。

$$M^* = \text{argmin}\, \mathcal{L}(M(K); V)$$

The Core Building Block: Associative Memory | 核心建構單元：聯想記憶

An operator **M** that learns to map a set of keys (K) to a set of values (V) by minimizing an objective.
一個運算子 **M**，透過最小化目標函數，學習將一組鍵 (K) 映射到一組值 (V)。

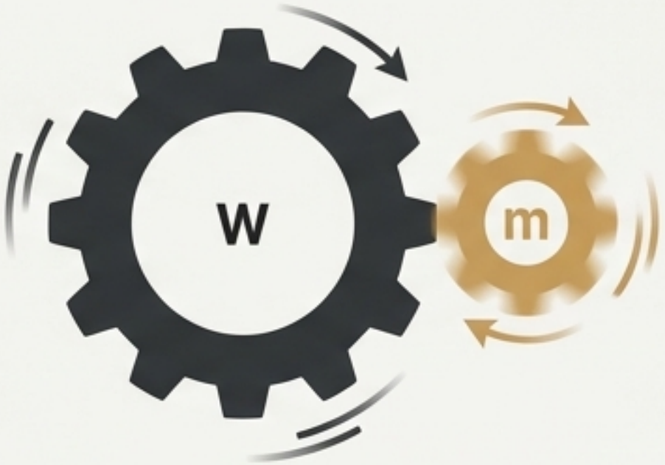
$$M^* = \text{argmin}\, \mathcal{L}(M(K); V)$$

In NL, **all elements** of a model—from attention to the optimizer—are treated as **associative memory systems** that learn by **compressing their context flow**. 在 NL 框架下，模型的所有元素——從注意力到優化器——都被視為透過**壓縮其脈絡流**來學習的聯想記憶系統。

Deconstruction: Models as Nested Optimization Problems | ### 解構：視模型為巢狀優化問題

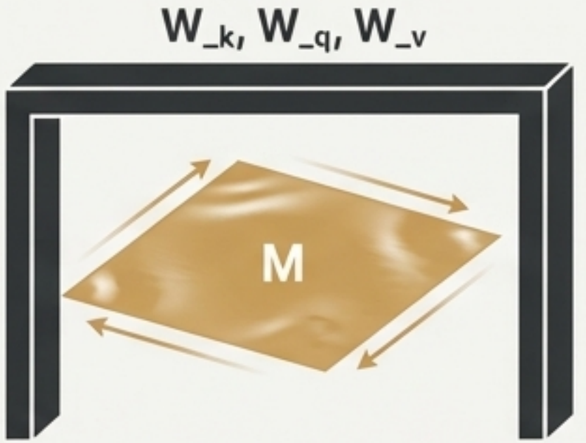
NL reveals that seemingly simple training processes are already multi-level optimization problems.
NL 揭示了看似簡單的訓練過程，實際上已是多層次的優化問題。

Example 1: Training an MLP with Momentum | 案例一：訓練帶有動量的 MLP



- **Level 1 (Outer Loop):** Model weights **W** are updated (slow).
  層級 1 (外迴圈): 模型權重 **W** 被更新 (慢速)。
- **Level 2 (Inner Loop):** The momentum term **m** is an associative memory compressing past gradients (fast).
  層級 2 (內迴圈): 動量項 **m** 是一個壓縮過去梯度的聯想記憶體 (快速)。

Example 2: Linear Attention | 案例二：線性注意力機制



- **Level 1 (Outer Loop):** Projection matrices **W**_k, **W**_q, **W**_v are trained (slow).
  層級 1 (外迴圈): 投影矩陣 **W**_k, **W**_q, **W**_v 被訓練 (慢速)。
- **Level 2 (Inner Loop):** The attention matrix **M** is updated with each new token (fast).
  層級 2 (內迴圈): 注意力矩陣 **M** 隨每個新 token 更新 (快速)。

This decomposition is based on **update frequency (更新頻率)**, creating a hierarchy of learning speeds.

Contribution 1: Optimizers as Learning Modules

貢獻一：視優化器為學習模組

NL reveals that well-known gradient-based optimizers are themselves learning modules. NL 揭示了著名的梯度優化器本身就是學習模組。

- **SGD with Momentum**: A 2-level system where the momentum term is a **key-less associative memory** that stores a history of gradients. 帶動量的 SGD 是一個雙層系統，動量項是一個儲存梯度歷史的**無鑰值聯想記憶體**。
- **Adam Optimizer**: An optimal associative memory for the model's gradients. Adam 優化器可被視為模型梯度的最佳聯想記憶體。

Building More Expressive Optimizers | 打造更具表達力的優化器

1. **More Expressive Association (關聯)**:: Use preconditioning to give memory meaningful values.

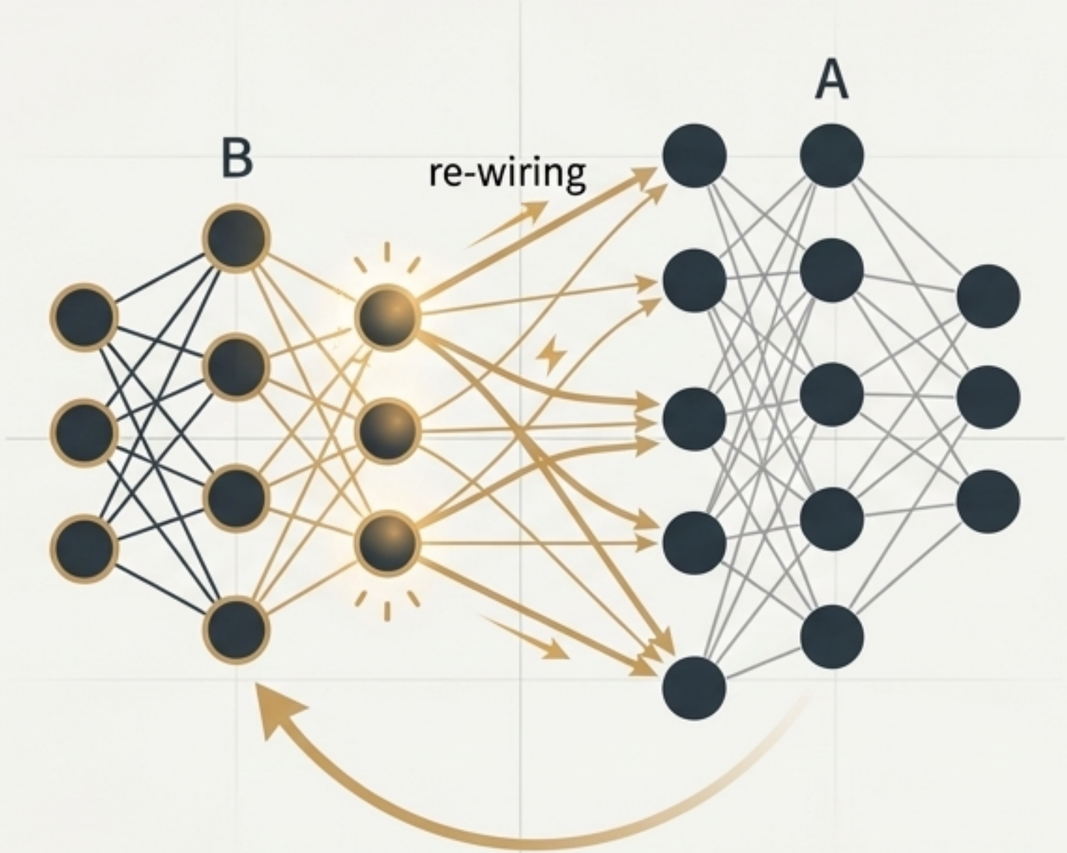
2. **More Expressive Objectives (目標)**:: Use L2 regression (Delta Rule) for the update.

3. **More Expressive Memory (記憶體)**:: Replace linear memory with a deep one (e.g., an MLP), resulting in **Deep Momentum Gradient Descent (DMGD)**.

Contribution 2: Self-Modifying Architectures | ### 貢獻二：自我修改架構

Leveraging NL, we can build models that learn how to modify themselves by learning their own update algorithm.
利用 NL 的洞見，我們可以建構能透過學習自身更新演算法來自我修改的模型。

- **Learnable Update Rules**: The update rules themselves can be parameterized and learned, rather than being fixed like backpropagation.
  可學習的更新規則：更新規則本身可以被參數化和學習，而非像反向傳播那樣固定不變。
- **Self-Referential Models**: One part of the network learns the algorithm that another part uses to update its weights.
  自我指涉模型：網路的一部分學習演算法，以供另一部分用來更新其權重。
- **Embodied in 'Titans'**: This concept is embodied in architectures like *Titans*, which learn to memorize and adapt at test time.
  "Titans" 架構的體現：此概念體現於 *Titans* 等架構中，它能在測試階段學習記憶與適應。



Self-Referential Update | 自我指涉更新

Inter SemiBold | Noto Sans TC

Contribution 3: The Continuum Memory System (CMS)

貢獻三：連續體記憶系統 (CMS)

CMS generalizes the traditional binary view of long-term vs. short-term memory. CMS 概括了傳統「長期 vs. 短期」記憶的二元觀點。

Instead of one fast memory (attention) and one slow memory (MLP), CMS proposes a **chain of memory modules**, each operating at a different frequency. CMS 不再是單一的快速記憶（注意力）和慢速記憶（MLP），而是提出一個**記憶模組鏈**，每個模組以不同頻率運作。

$$y_t = \text{MLP}^{(f_k)}(\ldots \text{MLP}^{(f_1)}(x_t)\ldots)$$

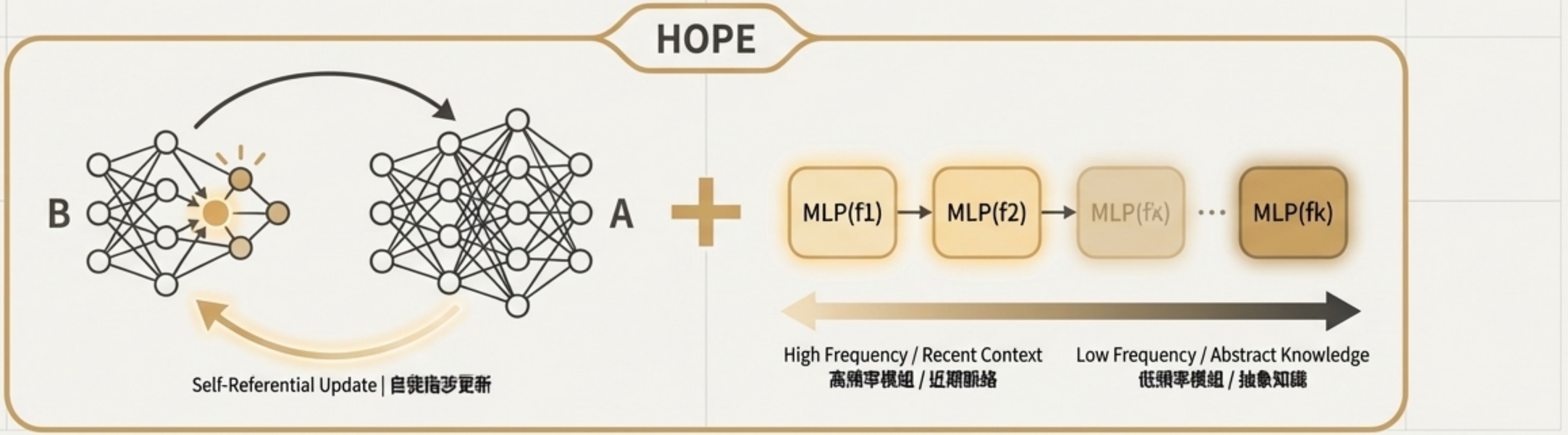
- **Faster-frequency blocks** (f_1) are updated more often, capturing recent context.
  高頻率模組 (f_1) 更新更頻繁，捕捉近期脈絡。
- **Slower-frequency blocks** (f_k) are updated infrequently, capturing abstract, long-term knowledge.
  低頻率模組 (f_k) 更新不頻繁，捕捉抽象的長期知識。

This creates a **spectrum of memory persistence and abstraction.** 這在模型內部創造了一個**記憶持久性與抽象度的光譜**。

Introducing HOPE: A Self-Referential Learning Module | ### 介紹 HOPE：一個自我指涉的學習模組

HOPE (Hierarchical Online Processing Engine) is the novel architecture that combines the key contributions.
HOPE (層級式線上處理引擎) 是結合本文核心貢獻的全新架構。

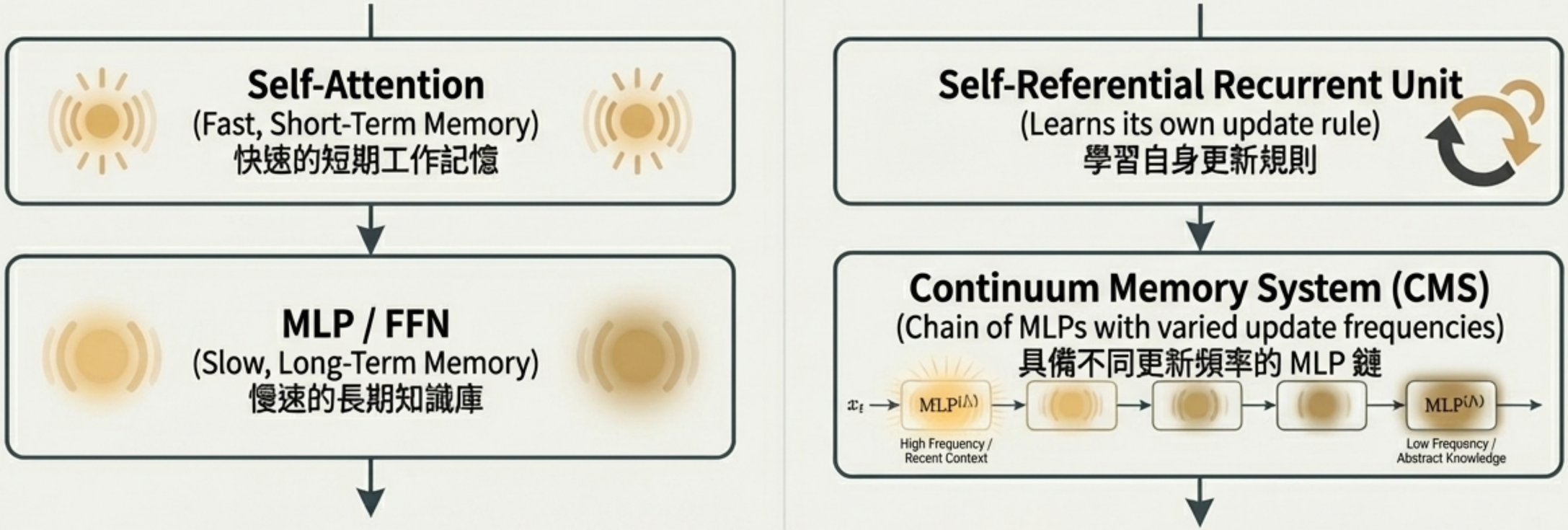
HOPE = Self-Referential Model + Continuum Memory System | HOPE = 自我指涉模型 + 連續體記憶系統



1. It uses a **self-referential sequence model** (based on Titans) as its working memory, allowing it to learn its own update rule from the context.
   它使用一個**自我指涉序列模型** (基於 Titans) 作為其工作記憶，使其能從脈絡中學習自己的更新規則。
2. It replaces the standard Feed-Forward Network (FFN) with the **Continuum Memory System (CMS)**, creating multiple levels of persistent memory with different update frequencies. 它用**連續體記憶系統 (CMS)** 取代標準的前饋網路 (FFN)，創造出具有不同更新頻率的多層次持久記憶。

Architecture Comparison: HOPE vs. Transformer
架構比較：HOPE vs. Transformer

HOPE introduces a new dimension of depth—**nested levels**—compared to the "flat" architecture of a standard Transformer. 與標準 Transformer 的「扁平」架構相比，HOPE 引入了一個新的深度維度——**巢狀層級**。



*Clarity Note: Diagrams are simplified. Normalization and other components are omitted for clarity.
*註：為清晰起見，圖表經過簡化，省略了正規化等組件。

Experimental Validation: Setup | ### 實驗驗證：設定

HOPE was evaluated against strong baselines on a range of standard benchmarks.
HOPE 在一系列標準基準測試中，與強大的基線模型進行了評估。

Tasks | 任務

1. **Language Modeling:** WikiText-103 (ppl ↓), LAMBADA (ppl ↓, acc ↑)
   語言模型：
2. **Common-Sense Reasoning:** PIQA, HellaSwag, WinoGrande, ARC, SIQA, BoolQ (acc ↑)
   常識推理：

Baselines | 基線模型

Transformer++
Modern RNNs: RetNet, DeltaNet, Samba, Titans

Scales | 規模

Models were compared at **340M**, **760M**, and **1.3B** parameter counts.
模型在 **3.4億**、**7.6億** 和 **13億** 參數規模下進行比較。

Key Results & Performance
關鍵結果與效能

HOPE demonstrates superior or highly competitive performance across all scales and tasks.
HOPE 在所有規模與任務上均展現出卓越或極具競爭力的效能。

Performance Highlights (1.3B Parameter Models)
效能亮點 (13億參數模型)

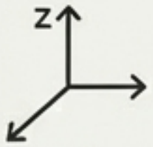
Model	LMB. ppl ↓	Avg. Acc ↑
Transformer++	18.32	52.25
RetNet	17.27	52.02
DeltaNet	16.88	52.14
Samba*	13.29	54.00
Titans (LMM)	11.41	56.82
HOPE (ours)	**11.63**	**57.23**

- HOPE consistently outperforms Transformer baselines and other modern recurrent architectures.
  HOPE 的表現持續優於 Transformer 基線及其他現代循環架構。
- It achieves the **highest average accuracy** on reasoning tasks, demonstrating the benefit of its dynamic, multi-level design.
  它在推理任務上取得了**最高的平均準確率**，證明其動態、多層級設計的優勢。

Conclusion: Beyond Stacking Layers
結論：超越堆疊層數

Nested Learning (NL) offers a new lens to understand and design ML models, moving beyond the simple illusion of depth created by stacking layers.
巢狀學習 (NL) 提供了一個新視角來理解和設計機器學習模型，超越了僅由堆疊層數創造的深度幻象。

Key Takeaways | 核心觀點



1. **A New Dimension:**
   NL introduces "levels" (層級) of optimization as a new dimension for designing more expressive models.

2. **White-Box Understanding:**
   It provides a transparent, mathematical framework for decomposing existing architectures.
   它提供了一個透明的數學框架來解構現有架構。

3. **Continual Learning:**
   By modeling online memory consolidation, NL offers a principled path toward models that can continually learn and adapt.
   透過模擬線上記憶鞏固，NL 為能持續學習的模型提供了路徑。

4. **HOPE as Validation:**
   The HOPE architecture validates these principles, achieving state-of-the-art results.
   HOPE 架構以其頂尖的成果驗證了這些原則。

Thank You

謝謝聆聽

Presenter: Chang, Chia-Kai (張家愷)

**GitHub:* github.com/Chiakai-Chang/PPTPlaner

**LinkedIn:* linkedin.com/in/chiakai-chang-htciu

Email: lotifv@gmail.com